

A Computer-Assisted Proof That $C_{1a} \geq \frac{7}{5}$

Jai Bajaj

Abstract

We prove that the autoconvolution constant

$$C_{1a} = \inf_f \frac{\|f * f\|_{L^\infty}}{(\int f)^2},$$

taken over nonnegative functions supported on $(-\frac{1}{4}, \frac{1}{4})$, satisfies $C_{1a} \geq \frac{7}{5}$. The proof reduces the infinite-dimensional optimization to a finite computation via a discretization of the mass simplex, building on the framework of Cloninger and Steinerberger [CS17], combined with a multiscale branch-and-prune cascade that eliminates all candidate configurations. The analytic reduction is formalized parametrically in Lean 4; the new headline bound depends additionally on a finite computational certificate, produced by the cascade described in Sections 5–6, asserting that every terminal-resolution composition has a window whose test value exceeds the corrected threshold.

2020 Mathematics Subject Classification. 11B83, 42A85, 68V20.

Keywords. Autoconvolution, Sidon sets, branch-and-prune, computer-assisted proof, Lean formalization.

1. Introduction

1.1. The Constant C_{1a}

¹

Let

$$\mathcal{F} = \left\{ f \in L^1(\mathbb{R}) : f \geq 0, \text{ supp}(f) \subseteq \left(-\frac{1}{4}, \frac{1}{4}\right), \int_{\mathbb{R}} f > 0 \right\}.$$

The *autoconvolution* of f is

$$(f * f)(t) = \int_{-\infty}^{\infty} f(x) f(t - x) dx.$$

Since $f \in L^1(\mathbb{R})$ has compact support, $f * f$ is continuous, nonnegative, and supported on $[-\frac{1}{2}, \frac{1}{2}]$.

Definition 1.1. The *autoconvolution ratio* is

$$R(f) = \frac{\|f * f\|_{L^\infty}}{(\int f)^2},$$

and the *autoconvolution constant* is

$$C_{1a} = \inf_{f \in \mathcal{F}} R(f).$$

¹All source code, the Lean 4 formalization, and the computational certificate are available at the accompanying repository.

Bound	Value	Reference
Earlier lower bounds	$\geq 1.2628, \geq 1.2748$	Martin & O’Bryant [MO07, MO09]
Published lower bound	≥ 1.2802	Cloninger & Steinerberger [CS17]
Cascade lower bound	$\geq \frac{7}{5} = 1.4$	This work
Published upper bound	≤ 1.50992	Matolcsi & Vinuesa [MV10]
Improved upper bound	≤ 1.50988	Boyer & Li [BL26]

Table 1: Published bounds on C_{1a} in the normalization used here, together with the present result. Numerical values track the progression $1 < 1.182778 < 1.2628 < 1.2748 < 1.2802 < \frac{7}{5} < 1.50988 < \pi/2$ in this normalization.

By homogeneity, $R(\lambda f) = R(f)$ for every $\lambda > 0$, so one may normalize $\int f = 1$ without loss of generality. The Lean definition `autoconvolution_ratio` uses the essential supremum via `eLpNorm`; for $f \in \mathcal{F}$ this agrees with the pointwise maximum because $f * f$ is continuous.

Why this constant matters. C_{1a} is connected to the asymptotic theory of Sidon and generalized Sidon sets in additive combinatorics [ET41, MO06, CRV10, MO09, CV08, CS17]. A *Sidon set* is a set of integers where all pairwise sums are distinct; equivalently, the “representation function” of the sumset has values at most 2 everywhere except at doubled elements. The constant C_{1a} governs how efficiently mass can be distributed to minimize the peak of the autoconvolution — the continuous analog of controlling pairwise-sum collisions.

1.2. Known Bounds and the Present Result

The *upper bound* comes from exhibiting explicit functions whose autoconvolution peak is small. This is a constructive task: find a good f and compute its peak. Boyer and Li [BL26] have recently improved the upper bound of Matolcsi–Vinuesa [MV10]; an informal catalog of historical and current bounds is maintained by Tao [Tao].

The *lower bound* is fundamentally harder: one must show that *no* function can achieve a peak below the threshold, which requires exhaustive elimination of all candidate configurations.

The published lower bounds of Martin–O’Bryant [MO07, MO09], Matolcsi–Vinuesa [MV10], and Cloninger–Steinerberger [CS17] give the progression

$$1 < 1.182778 < 1.2628 < 1.2748 < 1.2802$$

in the present normalization, while the older upper bound $\pi/2$ comes from Schinzel–Schmidt [SS02]. A related extremal problem in additive combinatorics, Erdős’ minimum overlap problem, is treated by analogous continuous-optimization techniques in White [Whi23].

1.3. Proof Strategy at a High Level

The proof has two components: a continuous reduction and a finite certificate. The continuous reduction is formalized in Lean; the finite certificate is produced by an exhaustive branch-and-prune computation.

At a high level, the reduction proceeds in three steps:

- (i) **Reduce** the infinite-dimensional optimization over functions to a family of windowed quadratic expressions in bin masses.
- (ii) **Discretize** the normalized bin masses by a specific cumulative-floor map, producing a composition $c \in \mathbb{N}^{2n}$ of total mass m together with a quantified per-window error term.
- (iii) **Certify** that every terminal composition at $(n, m) = (64, 20)$ is eliminated by some window whose test value exceeds the corrected threshold.

The terminal statement is verified by a dyadic cascade from $d = 4$ to $d = 128$, using reversal symmetry, asymmetry pruning, and the window-dependent correction term.

Theorem 1.2 (Main result). *Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be nonnegative with $\text{supp}(f) \subseteq (-\frac{1}{4}, \frac{1}{4})$ and $\int f > 0$. If $\|f * f\|_{L^\infty} < \infty$, then*

$$\frac{\|f * f\|_{L^\infty}}{(\int f)^2} \geq \frac{7}{5}.$$

*The case $\|f * f\|_{L^\infty} = \infty$ is vacuous for any finite lower bound, so $C_{1a} \geq \frac{7}{5} = 1.4$ holds unconditionally.*

The analytic reduction underlying this theorem is formalized in Lean 4 as a parametric chain in `Sidon/Proof/FinalResult.lean`; the current verified instantiation certifies the Cloninger–Steinerberger baseline $C_{1a} \geq \frac{32}{25}$ as `autoconvolution_ratio_ge_32_25` and serves as an end-to-end soundness check of the Lean development (see Section 9). The proof is assembled in Section 7 from the continuous reduction (Sections 2–4), the pruning machinery (Sections 5–6), and the terminal computational certificate produced by the cascade.

2. From Functions to Bin Averages

Our first order of business is to replace the infinite-dimensional optimization — over all nonnegative functions — with something finite-dimensional and computable.

2.1. The Partition

Fix a positive integer n and write $d = 2n$. The formalization uses the half-open partition

$$I_i = \left[-\frac{1}{4} + \frac{i}{4n}, -\frac{1}{4} + \frac{i+1}{4n} \right), \quad i = 0, \dots, d-1.$$

Each bin has width $\frac{1}{4n}$, and the family $\{I_i\}_{i=0}^{d-1}$ partitions $[-\frac{1}{4}, \frac{1}{4})$.

Figure 1 illustrates the support partition and the induced vectors of bin masses and bin averages.

2.2. Bin Masses and Bin Averages

Assume now that $f \in \mathcal{F}$ and $\int f = 1$. Define the *bin masses*

$$\mu_i = \int_{I_i} f(x) dx \geq 0, \quad i = 0, \dots, d-1,$$

and the associated *bin averages*

$$a_i = 4n \mu_i.$$

Since $|I_i| = \frac{1}{4n}$, the quantity a_i is the average value of f on I_i whenever f is constant on that bin.

Lemma 2.1 (Partition of mass). *For the normalized function f , the bin masses satisfy*

$$\sum_{i=0}^{d-1} \mu_i = 1, \quad \sum_{i=0}^{d-1} a_i = 4n.$$

Proof. The half-open bins partition $[-\frac{1}{4}, \frac{1}{4})$, while $\text{supp}(f) \subseteq (-\frac{1}{4}, \frac{1}{4})$ excludes the endpoints. Thus the sum of the bin integrals is $\int f = 1$, and the second identity is obtained by multiplying by $4n$. $\square$

The Lean counterpart is the lemma `sum_bin_masses_eq_one`.

Support partition and bin averages

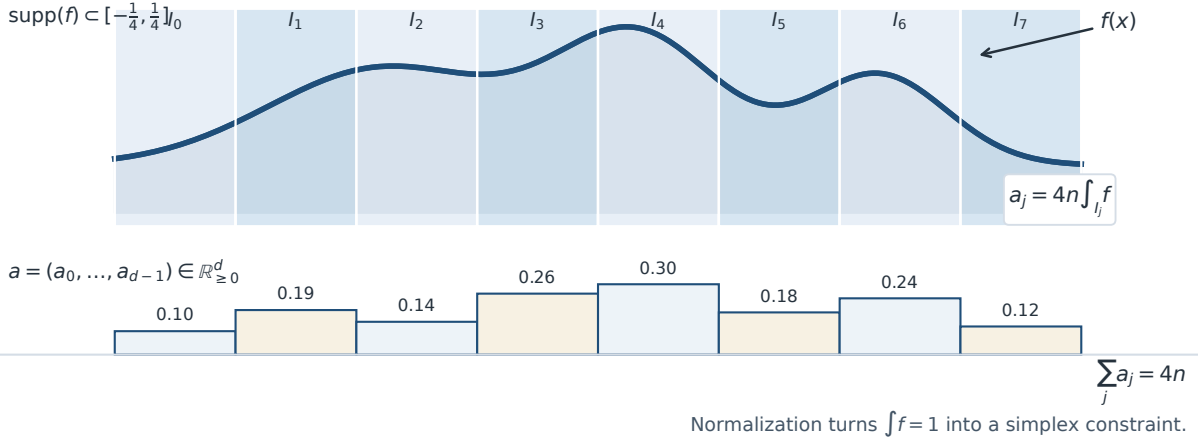


Figure 1: A nonnegative profile supported on $[-\frac{1}{4}, \frac{1}{4}]$, together with the half-open partition into equal bins. The upper panel shows the support decomposition, while the lower panel records the induced averages $a_i = 4n \mu_i$ and the simplex identity $\sum_i a_i = 4n$ after normalizing $\int f = 1$.

Definition 2.2. The *normalized mass simplex* and the *bin-average simplex* are

$$\Delta_n = \left\{ \mu \in \mathbb{R}_{\geq 0}^d : \sum_{i=0}^{d-1} \mu_i = 1 \right\},$$

$$A_n = \left\{ a \in \mathbb{R}_{\geq 0}^d : \sum_{i=0}^{d-1} a_i = 4n \right\} = 4n \Delta_n.$$

The vector $\mu = (\mu_0, \dots, \mu_{d-1})$ is the object discretized in Lean, whereas the windowed quadratic forms are most naturally written in the rescaled coordinates $a = 4n \mu$.

2.3. Window Lower Bound for the Ratio

The next lemma passes from the continuous problem to the discrete windowed expressions of Cloninger and Steinerberger [CS17, Lemma 1].

Lemma 2.3 (Per-window test-value bound). *Let $f \in \mathcal{F}$ satisfy $\int f = 1$, and let $a_i = 4n \int_{I_i} f$. Then for every pair of integers $\ell \geq 2$ and $s_{10} \geq 0$,*

$$\|f * f\|_{L^\infty} \geq \frac{1}{4n\ell} \sum_{s=s_{10}}^{s_{10}+\ell-2} \sum_{\substack{0 \leq i, j \leq d-1 \\ i+j=s}} a_i a_j.$$

Proof sketch. Write $f_i = f \mathbf{1}_{I_i}$. Then $f = \sum_i f_i$ and $(f * f)(t) = \sum_{i,j} (f_i * f_j)(t)$. The support of $f_i * f_j$ is contained in $I_i + I_j$, an interval of length $\frac{1}{2n}$ whose location depends only on $i + j$. Fixing the sum-index window $s_{10} \leq i + j \leq s_{10} + \ell - 2$ selects $\ell - 1$ consecutive sum-indices. Their Minkowski-sum intervals are translates by $\frac{1}{4n}$ of a fixed interval of length $\frac{1}{2n}$, so their union is a single interval of length $(\ell - 2) \cdot \frac{1}{4n} + \frac{1}{2n} = \frac{\ell}{4n}$. The total L^1 mass contributed by these terms is

$$\sum_{s=s_{10}}^{s_{10}+\ell-2} \sum_{i+j=s} \frac{a_i}{4n} \frac{a_j}{4n}.$$

Applying the averaging inequality $\|h\|_{L^\infty} \geq |J|^{-1} \int_J h$ to this interval gives the claimed lower bound. $\square$

The Lean counterpart is `continuous_test_value_le_ratio`, stated per window; the maximized version follows by taking the supremum over all admissible (ℓ, s_{10}) .

Geometrically, the products $a_i a_j$ quantify how much mass from bins i and j can contribute to the autoconvolution. Narrow windows carry the favorable prefactor $\frac{1}{4n\ell}$ but capture fewer pairs; wider windows capture more pairs but average over a longer interval. The branch-and-prune computation exploits this trade-off.

3. Discretization of the Simplex

Having reduced the problem to minimizing over the continuous simplex A_n , we face a new obstacle: A_n is uncountable. To bring the problem within reach of a computer, we discretize the normalized mass simplex Δ_n . Regular simplex grids are standard in finite-dimensional optimization [BGY14]; here the key point is that the formal proof uses a particular lattice point selected by a cumulative-floor rule.

3.1. The Lattice

Fix a positive integer m . In normalized mass coordinates, the natural grid has spacing $1/m$.

Definition 3.1. Fix a positive integer m (the “grid resolution”). The *discrete lattice* is

$$B_{n,m} = \left\{ b \in \left(\frac{1}{m}\mathbb{N}\right)^d \left| \sum_{j=0}^{d-1} b_j = 1 \right. \right\},$$

where each b_j is a nonnegative multiple of $\frac{1}{m}$.

Equivalently, writing $c_i = mb_i$, the lattice consists of nonnegative integer vectors $c = (c_0, \dots, c_{d-1})$ with $\sum c_j = m$. These are the *compositions* of the integer m into d nonnegative parts.

Figure 2 shows this discretization on a two-dimensional toy simplex, where a continuous point is rounded to a nearby lattice point.

For the formal argument, however, one needs a specific lattice point, not merely an existence statement.

Definition 3.2 (Canonical discretization). Let $\mu \in \Delta_n$ and define the cumulative masses

$$M(k) = \sum_{i < k} \mu_i, \quad 0 \leq k \leq d.$$

Set

$$D(k) = \lfloor m M(k) \rfloor, \quad 0 \leq k \leq d,$$

and define the integer composition $c = (c_0, \dots, c_{d-1})$ by

$$c_i = D(i+1) - D(i) \quad (0 \leq i \leq d-2), \quad c_{d-1} = m - D(d-1).$$

The associated normalized lattice point is $w_i = c_i/m$.

More precisely, the Lean definition `canonical_discretization` is applied to the function f and writes

$$D_f(k) = \left\lfloor m \frac{\sum_{i < k} \mu_i}{\sum_j \mu_j} \right\rfloor$$

in terms of the bin masses $\mu_i = \int_{I_i} f$. By Lemma 2.1, the denominator is 1 once $\int f = 1$, recovering Definition 3.2.

Discretization of the simplex

One continuous point is rounded to a nearby grid point.

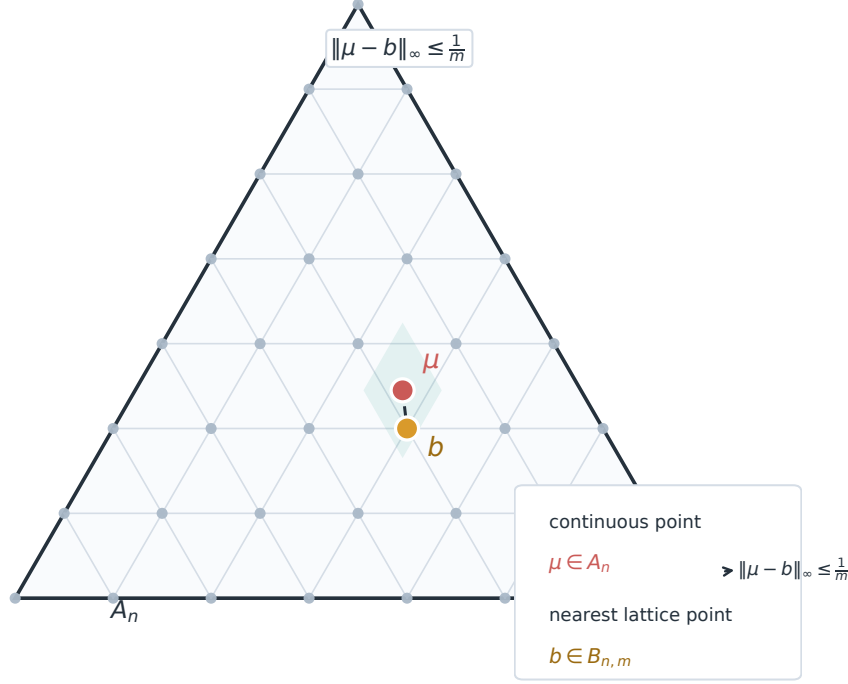


Figure 2: A three-coordinate slice of the normalized simplex. The point μ is replaced by a nearby lattice point $b \in B_{n,m}$; in the formal proof this representative is chosen by the cumulative-floor map described below.

Proposition 3.3 (Basic properties of the canonical discretization). *The composition c from Definition 3.2 satisfies*

$$\sum_{i=0}^{d-1} c_i = m, \quad \left| \frac{c_i}{m} - \mu_i \right| \leq \frac{1}{m} \quad \text{for each } i.$$

Proof sketch. The sum identity is telescoping:

$$\sum_{i=0}^{d-2} (D(i+1) - D(i)) + m - D(d-1) = m.$$

For the coordinate error, write

$$\frac{c_i}{m} - \mu_i = \left(\frac{D(i+1)}{m} - M(i+1) \right) - \left(\frac{D(i)}{m} - M(i) \right),$$

and use the elementary floor bounds $-1 < \lfloor x \rfloor - x \leq 0$. In Lean these statements are formalized by `canonical_discretization_sum_eq_m` together with the coordinatewise estimate proved in `Sidon/DiscretizationError.lean`. $\square$

3.2. The Discretization Error Bound

Define the set of *contributing bins* for a window (ℓ, s_{lo}) by

$$\mathcal{B}_{n,\ell,s_{\text{lo}}} = \{i \in \{0, \dots, d-1\} : \exists j \in \{0, \dots, d-1\}, s_{\text{lo}} \leq i+j \leq s_{\text{lo}} + \ell - 2\}.$$

Equivalently,

$$\mathcal{B}_{n,\ell,s_{\text{lo}}} = \{i : \max(0, s_{\text{lo}} - (d-1)) \leq i \leq \min(d-1, s_{\text{lo}} + \ell - 2)\},$$

which is the content of the Lean theorem `contributing_bins_iff`.

The next estimate is the exact per-window bound used later in the main theorem.

Lemma 3.4 (Per-window discretization error). *Let $f \in \mathcal{F}$ satisfy $\int f = 1$, let c be the canonical discretization from Definition 3.2, and set $w_i = c_i/m$. For a fixed window (ℓ, s_{lo}) , write*

$$W_{\text{int}} = \sum_{i \in \mathcal{B}_{n,\ell,s_{\text{lo}}}} c_i, \quad W = \sum_{i \in \mathcal{B}_{n,\ell,s_{\text{lo}}}} w_i = \frac{W_{\text{int}}}{m}.$$

Then

$$\text{TV}_{n,m}(c; \ell, s_{\text{lo}}) - \text{TV}_n^{\text{cont}}(f; \ell, s_{\text{lo}}) \leq \frac{4n}{\ell} \left(\frac{1}{m^2} + \frac{2W}{m} \right),$$

where

$$\text{TV}_{n,m}(c; \ell, s_{\text{lo}}) = \frac{1}{4n\ell} \sum_{s=s_{\text{lo}}}^{s_{\text{lo}}+\ell-2} \sum_{i+j=s} \left(\frac{4n}{m} c_i \right) \left(\frac{4n}{m} c_j \right),$$

and $\text{TV}_n^{\text{cont}}(f; \ell, s_{\text{lo}})$ is the corresponding continuous expression from Lemma 2.3.

Proof sketch. Write $w_i = \mu_i + \delta_i$, so the discrete and continuous test values differ by terms linear and quadratic in (δ_i) . The quadratic part is bounded by $\frac{4n}{\ell} \cdot \frac{1}{m^2}$ from the pointwise estimate $|\delta_i| \leq \frac{1}{m}$. The linear part is the load-bearing step. Proposition 3.3 shows the partial sums $D(k)/m - M(k) = \sum_{i < k} \delta_i$ lie in $(-1/m, 0]$, so for any range of consecutive indices the cumulative δ -error telescopes to at most $1/m$ in absolute value. Applied to the contributing bin range $\mathcal{B}_{n,\ell,s_{\text{lo}}}$ and combined with $|\mu_i| \leq W$, the linear cross-term is bounded by $\frac{4n}{\ell} \cdot \frac{2W}{m}$. This is the estimate proved in Lean as `correction_term_bound_w_refined` (and its CS variant `correction_term_bound_cs`). $\square$

Corollary 3.5 (Global correction bound). *For every admissible window one has*

$$\text{TV}_{n,m}(c; \ell, s_{\text{lo}}) - \text{TV}_n^{\text{cont}}(f; \ell, s_{\text{lo}}) \leq 2n \left(\frac{2}{m} + \frac{1}{m^2} \right),$$

because $W \leq 1$ and $\ell \geq 2$.

3.3. The Refined (Dynamic) Correction

The global correction bound is convenient for estimates, but the formal proof and the certificate computation use the sharper per-window threshold.

Theorem 3.6 (Dynamic threshold soundness). *Let c be the canonical discretization of a normalized function $f \in \mathcal{F}$ at parameters (n, m) , fix a window (ℓ, s_{lo}) with $\ell \geq 2$, and let W be as in Lemma 3.4. If*

$$\text{TV}_{n,m}(c; \ell, s_{\text{lo}}) > c_{\text{target}} + \frac{4n}{\ell} \left(\frac{1}{m^2} + \frac{2W}{m} \right),$$

then $R(f) \geq c_{\text{target}}$.

Proof. Combine Lemma 2.3 with Lemma 3.4. The first gives $R(f) \geq \text{TV}_n^{\text{cont}}(f; \ell, s_{\text{lo}})$, and the second shows that the discrete test value exceeds the continuous one by at most the correction term displayed above. $\square$

The Lean counterpart is the theorem `dynamic_threshold_sound_w_refined`, with a CS-form variant `dynamic_threshold_sound_cs` that uses the global correction term.

Remark 3.7 (Integer-coordinate form). Substituting $W = W_{\text{int}}/m$ into Theorem 3.6 rewrites the threshold as

$$c_{\text{target}} + \frac{4n}{\ell} \frac{1 + 2W_{\text{int}}}{m^2}.$$

This is the exact test-value threshold that reappears in the terminal certificate at $(n, m) = (64, 20)$.

4. The Windowed Test Value

With the discretization machinery in hand, we can describe precisely what the algorithm computes at each lattice point.

4.1. Definition

Definition 4.1. For a mass vector $a = (a_0, \dots, a_{d-1})$ with $a_i \geq 0$ and $\sum a_i = 4n$, the *autoconvolution* is the vector of length $2d - 1$:

$$\text{conv}[s] = \sum_{\substack{i+j=s \\ 0 \leq i, j \leq d-1}} a_i a_j, \quad s = 0, 1, \dots, 2(d-1).$$

The *test value* is

$$\text{tv}(a) = \max_{\substack{2 \leq \ell \leq 2d \\ 0 \leq s_{\text{lo}} \leq 2(d-1) - (\ell-2)}} \frac{1}{4n\ell} \sum_{s=s_{\text{lo}}}^{s_{\text{lo}}+\ell-2} \text{conv}[s].$$

In words: compute the autoconvolution, slide a window across it that covers $\ell - 1$ consecutive entries, normalize by $4n\ell$, and take the maximum. The parameter ℓ matches the convention of Cloninger–Steinerberger [CS17]: the corresponding Minkowski-sum interval has length $\ell/(4n)$, even though the window contains $\ell - 1$ entries.

(The Lean definition `max_test_value` lets s_{lo} range over all values $0, \dots, 2d - 1$; positions where the window extends past the support of `conv` contribute only zeros and do not change the maximum.)

Remark 4.2 (Integer composition coordinates). The code stores compositions $c = (c_0, \dots, c_{d-1}) \in \mathbb{N}^d$ with $\sum_i c_i = m$ and recovers the bin-average vector by

$$a_i = \frac{4n}{m} c_i.$$

All pruning formulas below are stated in these integer coordinates.

4.2. Efficient Computation via Prefix Sums

The test value can be computed efficiently using prefix sums. Let $P[s] = \sum_{k=0}^s \text{conv}[k]$ be the prefix sum of the autoconvolution. Then the windowed sum from s_{lo} to s_{hi} is:

$$\sum_{s=s_{\text{lo}}}^{s_{\text{hi}}} \text{conv}[s] = P[s_{\text{hi}}] - P[s_{\text{lo}} - 1] \quad (\text{with } P[-1] = 0).$$

The autoconvolution costs $O(d^2)$ per lattice point (an FFT-based method would give $O(d \log d)$ but is slower in practice for the moderate d values used here). The prefix-sum construction and the complete window scan each cost $O(d^2)$.

4.3. Reversal Symmetry

A key structural observation cuts the search space in half. If we reverse the order of the bins, the autoconvolution peak does not change — intuitively, because convolution does not care which direction we label the support.

Theorem 4.3 (Autoconvolution Reversal Symmetry). *For any mass vector a , $\text{tv}(a) = \text{tv}(\text{rev}(a))$, where $\text{rev}(a) = (a_{d-1}, \dots, a_0)$.*

Proof. The argument has three moving parts.

Step 1: Autoconvolution reversal. Let $a' = \text{rev}(a)$, so $a'_i = a_{d-1-i}$. Then:

$$\text{conv}'[s] = \sum_{i+j=s} a'_i a'_j = \sum_{i+j=s} a_{d-1-i} a_{d-1-j}.$$

Substituting $i' = d - 1 - i$, $j' = d - 1 - j$ (so $i' + j' = 2(d - 1) - s$):

$$\text{conv}'[s] = \text{conv}[2(d - 1) - s].$$

That is, the autoconvolution of the reversal is the reversal of the autoconvolution.

Step 2: Window-sum bijection. A window sum of width $w = \ell - 1$ starting at s_0 transforms under reversal to a window sum of the same width starting at $2(d - 1) - s_0 - w + 1$:

$$\begin{aligned} \sum_{s=s_0}^{s_0+w-1} \text{conv}'[s] &= \sum_{s=s_0}^{s_0+w-1} \text{conv}[2(d - 1) - s] \\ &= \sum_{s'=2(d-1)-s_0-w+1}^{2(d-1)-s_0} \text{conv}[s']. \end{aligned}$$

This is a window of the same width w , so the multiset of all window sums at width w is identical for conv and conv' .

Step 3: Normalization preserves the maximum. The normalization $\frac{1}{4n\ell}$ depends only on ℓ , not the window position. Since the multiset of window sums is identical at each width, $\text{tv}(a) = \text{tv}(a')$. $\square$

Corollary 4.4 (Canonical Filter). *It suffices to check only canonical integer compositions c with $c \leq \text{rev}(c)$ lexicographically. The associated bin-average vectors have the same test value, so this halves the search space (up to palindromes).*

Figure 3 depicts the reversal orbit of a composition and the lexicographic canonicalization used to choose one representative from each non-palindromic pair.

Even after exploiting reversal symmetry, the number of integer states to check — roughly $\binom{m+d-1}{d-1}/2$ — grows rapidly with d and m . To keep the computation tractable, we need ways to rule out large swathes of the simplex without computing the full test value.

5. Pruning Strategies

5.1. Asymmetry Pruning

The following bound depends on f only through the mass on each half of the support. Despite its coarseness, this single test eliminates a majority of configurations at every cascade level.

Reversal symmetry and canonicalization

$\text{tv}(a) = \text{tv}(\text{rev}(a))$ reduces the search to one representative per orbit.
mirror symmetry

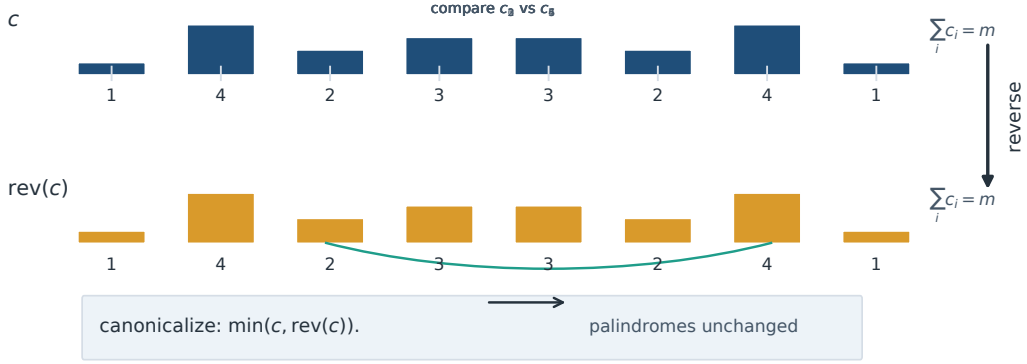


Figure 3: Reversal symmetry of the test value and the canonicalization rule $c \mapsto \min(c, \text{rev}(c))$. Palindromic compositions are fixed by the involution, while non-palindromic pairs contribute only one canonical representative to the search.

Theorem 5.1 (Asymmetry Bound). *Let $f \geq 0$ with $\text{supp}(f) \subseteq (-\frac{1}{4}, \frac{1}{4})$ and $\int f = 1$. Let $L = \int_{-1/4}^0 f$ be the left-half mass. Then:*

$$\|f * f\|_{L^\infty} \geq 2L^2.$$

Proof. Consider the left-left convolution:

$$(f * f)(t) = \int f(x) f(t-x) dx \geq \int_{-1/4}^0 f(x) f(t-x) \mathbf{1}_{[-1/4, 0]}(t-x) dx$$

for $t \in (-\frac{1}{2}, 0)$. By Fubini's theorem, the total mass of $f * f$ restricted to $(-\frac{1}{2}, 0)$ is at least L^2 (the left-left contribution). Since $|(-\frac{1}{2}, 0)| = \frac{1}{2}$:

$$\|f * f\|_{L^\infty} \geq \frac{L^2}{1/2} = 2L^2.$$

By symmetry, the same holds for the right-half mass $R = 1 - L$: $\|f * f\|_\infty \geq 2R^2$. Therefore $\|f * f\|_\infty \geq 2 \max(L^2, R^2)$. $\square$

Corollary 5.2 (Asymmetry Threshold). *If $2L^2 \geq c_{\text{target}}$, i.e., $L \geq \sqrt{c_{\text{target}}/2}$, then the configuration is ruled out. The threshold left-mass fraction is:*

$$p^* = \sqrt{c_{\text{target}}/2}.$$

For $c_{\text{target}} = \frac{7}{5}$, $p^* = \sqrt{0.7} \approx 0.8367$.

Remark 5.3 (Role inside the cascade). The asymmetry bound is a *direct* L^∞ bound on $f * f$ and does not go through the discretization framework of Lemma 3.4; in particular, for a step function represented by a composition c the left-half mass $L = \sum_{i < n} c_i/m$ is exact and is preserved by dyadic refinement. Inside the certificate computation, asymmetry therefore serves as an inexpensive early pruning rule before the window

scan. Together with reversal canonicalization (Section 4.3) and the single-bin cap (Section 6.3), it shrinks the active cascade by orders of magnitude, but the resulting Lean axiom records only the terminal window witnesses; the final analytic reduction depends solely on those witnesses.

5.2. Canonical Symmetry Exploitation

If c is an integer composition and $\tilde{c}_i = \frac{4n}{m}c_i$ is its associated bin-average vector, then Theorem 4.3 gives $\text{tv}(\tilde{c}) = \text{tv}(\text{rev}(c))$. This motivates the following notion of canonicity.

Definition 5.4. A composition c is *canonical* if $c \leq \text{rev}(c)$ lexicographically. That is, comparing c_0 with c_{d-1} , then c_1 with c_{d-2} , etc., c comes first (or they are equal, i.e., c is a palindrome).

Proposition 5.5 (Canonical Count). *For integer compositions of a fixed total mass M into d parts, the number of canonical compositions is $\frac{N+P}{2}$, where $N = \binom{M+d-1}{d-1}$ is the total count and P is the number of palindromes.*

Proof. Every composition falls into exactly one of three categories: palindromes (those equal to their own reversal), strictly canonical non-palindromes ($b < \text{rev}(b)$), and their reversals. Since reversal is an involution with no fixed points outside the palindromes, the second and third categories have equal size C . Counting gives $N = P + 2C$, so the canonical count is $P + C = (N + P)/2$. $\square$

Remark 5.6 (Loop-bound tightening for $d = 4$). For $b = (c_0, c_1, c_2, c_3)$ with total mass M and $c_3 = M - c_0 - c_1 - c_2$, canonicity ($b \leq \text{rev}(b)$) requires $c_0 \leq c_3$, giving three loop bounds:

- $c_0 \leq \lfloor M/2 \rfloor$ (if $c_0 > M/2$ then $c_3 < c_0$).
- $c_1 \leq M - 2c_0$ (need $c_2 \geq 0$ with $c_1 + c_2 \leq M - 2c_0$).
- $c_2 \leq M - 2c_0 - c_1$ (ensures $c_3 \geq c_0$).

Plus a palindrome filter: when $c_0 = c_3$, require $c_1 \leq c_2$.

For $d = 6$, analogous bounds apply: $c_0 \leq \lfloor M/2 \rfloor$ and $c_4 \leq M - 2c_0 - c_1 - c_2 - c_3$ (so that $c_5 = M - c_0 - c_1 - c_2 - c_3 - c_4 \geq c_0$), with palindrome filters $c_1 \leq c_4$ when $c_0 = c_5$ and $c_2 \leq c_3$ when both outer pairs are tied.

6. The Multiscale Cascade

Direct enumeration at the terminal resolution is intractable: even modest parameters yield billions of lattice points. The cascade starts at coarse resolution, eliminates configurations cheaply, and refines only those that survive.

6.1. Overview

- Step 1: Start coarse.** At level L_0 , enumerate all canonical compositions of m into $d_0 = 2n_0$ parts (e.g., $n_0 = 2, d_0 = 4$).
- Step 2: Prune.** For each composition, apply asymmetry pruning and then scan windows using the corrected dynamic threshold. If some window certifies the target bound, the composition is eliminated.
- Step 3: Refine survivors.** Each surviving composition at resolution d is refined to resolution $2d$ by *dyadic splitting*: each bin is split into two sub-bins.
- Step 4: Repeat.** Prune the refined compositions. Continue until either all are eliminated (proof complete) or a computational budget is reached.

Figure 4 summarizes the terminal certificate run, including the exact survivor counts at each dyadic level.

Multiscale cascade

Coarse levels prune cheaply; surviving branches are refined dyadically.

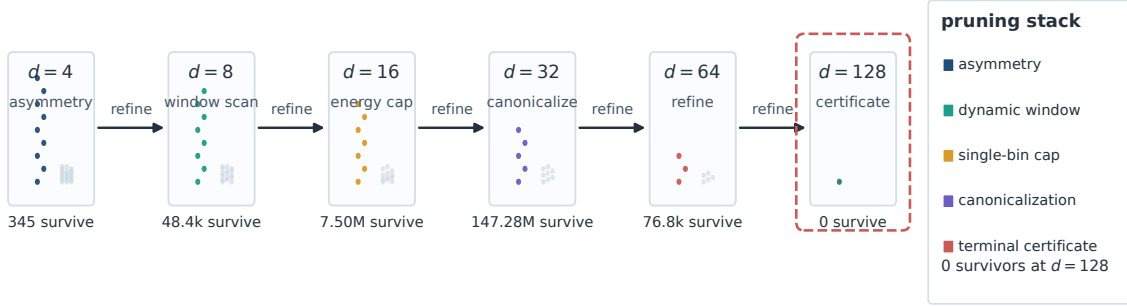


Figure 4: Stylized overview of the multiscale cascade. The annotations record the exact survivor counts from the terminal certificate run: 345 at $d = 4$, then 48,443, 7,499,382, 147,279,894, 76,829, and finally 0 at $d = 128$.

6.2. Dyadic Splitting (Refinement)

Given a parent composition $b = (b_0, \dots, b_{d-1})$ at resolution d , the refinement produces children $c = (c_0, c_1, \dots, c_{2d-1})$ at resolution $2d$, where:

$$c_{2i} + c_{2i+1} = b_i \quad \text{for each } i = 0, \dots, d-1,$$

with $c_{2i}, c_{2i+1} \geq 0$ and both being nonnegative integers.

6.3. Energy Cap

Many children produced by dyadic splitting are ruled out immediately by the single-bin diagonal window. The energy cap avoids generating them at all.

Recall that $a_j = \frac{4n_{\text{child}}}{m} c_j$ and the $\ell = 2$ diagonal window at position $2j$ gives

$$\text{tv}(a) \geq d_{\text{child}} \frac{c_j^2}{m^2}, \quad d_{\text{child}} = 2n_{\text{child}}.$$

This yields the test-value cap

$$x_{\text{cap}}^{\text{TV}} = \left\lfloor m \sqrt{\frac{c_{\text{target}} + 2n_{\text{child}} \left(\frac{2}{m} + \frac{1}{m^2} \right) + 10^{-9}}{d_{\text{child}}}} \right\rfloor.$$

The implementation also applies the sharper direct cap

$$x_{\text{cap}}^{\text{CS}} = \left\lfloor m \sqrt{\frac{c_{\text{target}}}{d_{\text{child}}}} \right\rfloor,$$

and uses $x_{\text{cap}} = \min(x_{\text{cap}}^{\text{TV}}, x_{\text{cap}}^{\text{CS}})$.

Proposition 6.1 (Energy Cap Soundness). *If a child has some bin $c_j > x_{\text{cap}}^{\text{CS}}$, then any nonnegative function with those child-bin masses already satisfies $\|f * f\|_{L^\infty} > c_{\text{target}}$.*

Proof. Let $M_j = c_j/m$ be the mass in the j th child bin. Restricting f to that bin gives a nonnegative function f_j supported on an interval of length $1/(2d_{\text{child}})$ (the child-bin width) with total mass M_j , hence $f_j * f_j$ is supported on an interval of length $1/d_{\text{child}}$ with total mass M_j^2 . Pointwise nonnegativity of f gives $(f * f)(t) \geq (f_j * f_j)(t)$, so applying the averaging inequality $\|h\|_{L^\infty} \geq |J|^{-1} \int_J h$ to $h = f_j * f_j$ over its support yields

$$\|f * f\|_{L^\infty} \geq \|f_j * f_j\|_{L^\infty} \geq \frac{M_j^2}{1/d_{\text{child}}} = d_{\text{child}} \frac{c_j^2}{m^2}.$$

If $c_j > m\sqrt{c_{\text{target}}/d_{\text{child}}}$, the right-hand side exceeds c_{target} , which proves the claim. $\square$

Remark 6.2 (Tighter Cauchy–Schwarz cap). The cap $x_{\text{cap}}^{\text{CS}}$ is tighter than $x_{\text{cap}}^{\text{TV}}$ and requires no discretization correction term; the implementation uses the minimum of the two.

6.4. Terminal Certificate

Theorem 6.3 (Recorded Terminal-Grid Certificate). *For every composition $c = (c_0, \dots, c_{127}) \in \mathbb{N}^{128}$ with $\sum_{i=0}^{127} c_i = 20$, let*

$$a_i = \frac{4 \cdot 64}{20} c_i, \quad i = 0, \dots, 127,$$

and let conv_a be the discrete autoconvolution of a . Then there exist integers $\ell \geq 2$ and s_{10} such that

$$\frac{1}{4 \cdot 64 \cdot \ell} \sum_{s=s_{10}}^{s_{10}+\ell-2} \text{conv}_a[s] > \frac{7}{5} + \frac{4 \cdot 64}{\ell} \left(\frac{1}{20^2} + \frac{2}{20^2} \sum_{i \in \mathcal{B}_{64,\ell,s_{10}}} c_i \right),$$

where $\mathcal{B}_{64,\ell,s_{10}}$ is the set of contributing bins for that window.

This is the computational claim imported into the Lean formalization as the axiom `cascade_all_pruned`. The dyadic cascade from $d = 4$ to $d = 128$ terminates with zero survivors at level L_5 ; the completeness properties in Section 8 explain why this terminal outcome implies the universal quantification above. The computation uses exact integer arithmetic for autoconvolutions and conservative floating-point rounding for threshold comparisons; no conclusion depends on floating-point correctness.

6.5. Canonical Handling Across Levels

At level L_0 , only canonical compositions are generated. At levels L_1 and beyond, the situation is more subtle:

- Children of a canonical parent P are *not* necessarily canonical.
- The reversal $\text{rev}(P)$ is *not* in the parent list (unless P is a palindrome), so its children are not generated directly.
- However, if C is a child of P , then $\text{rev}(C)$ is a child of $\text{rev}(P)$. Since $\text{tv}(C) = \text{tv}(\text{rev}(C))$ (Theorem 4.3), testing C suffices for both.

Therefore at levels ≥ 1 :

1. Generate *all* children of each parent (no canonical filter during generation).
2. Test each child against the dynamic threshold.
3. *Canonicalize* survivors: replace each with $\min(C, \text{rev}(C))$ lexicographically.
4. *Deduplicate*: remove duplicate canonical forms (children from different parents, or a palindrome parent producing both C and $\text{rev}(C)$).

Proposition 6.4 (Completeness of Canonical Handling). *This procedure tests every composition at the child resolution: if C' is any child-level composition and $P' = \text{parent}(C')$, then either P' or $\text{rev}(P')$ is in the parent list, and C' or $\text{rev}(C')$ is among the tested children.*

6.6. Gray-Code Fused Kernel

At deep cascade levels, each parent may have millions of children. The current CPU kernel enumerates the same child set as the naive Cartesian-product generator, but does so in mixed-radix Gray-code order and fuses generation, pruning, canonicalization, and survivor buffering into a single pass. Consecutive children differ in exactly one split coordinate, so the autoconvolution and tracked window mass can be updated locally instead of recomputed from scratch:

- **Incremental updates:** every Gray-code step changes one parent split, giving an $O(d)$ raw-convolution update and an $O(1)$ update of the tracked quick-check window mass.
- **Quick-check and ordered scans:** the previous killing window is retried first; if that fails, windows are scanned in a profile-guided order, but every admissible (ℓ, s_{10}) is still checked before a child is declared a survivor.
- **Lookup-table thresholds and staged survivor writes:** the conservative integer threshold is precomputed as a table in (ℓ, W_{int}) , and surviving rows are batched through an L1-resident staging buffer before they are flushed to the main output array.
- **Block skipping:** subtree pruning uses partial autoconvolutions of fixed prefixes, and the innermost Gray-code sweep can be fast-forwarded when a univariate range check proves that the same killing window eliminates the rest of that sweep.

7. The Complete Proof

7.1. Algorithm

Algorithm 1 Cascade Search For Terminal Coverage

Require: c_{target} , grid resolution m , starting half-bins n_0 , max levels L_{max}

```

1:  $d_0 \leftarrow 2n_0$ 
2: Generate all canonical compositions of  $m$  into  $d_0$  parts ▷ integer composition coordinates
3: Apply asymmetry pruning (threshold  $\sqrt{c_{\text{target}}/2}$ )
4: For every remaining composition, scan all windows and prune if some window exceeds the dynamic
   threshold from Theorem 3.6
5:  $\text{survivors}_0 \leftarrow$  unpruned compositions
6: for  $k = 1, 2, \dots, L_{\text{max}}$  do
7:    $d_k \leftarrow 2d_{k-1}; \quad n_k \leftarrow 2n_{k-1}$ 
8:   Compute the single-bin child cap  $x_{\text{cap}} = \min(x_{\text{cap}}^{\text{TV}}, x_{\text{cap}}^{\text{CS}})$ 
9:   for all parent  $P \in \text{survivors}_{k-1}$  do
10:    for all children  $C$  of  $P$  in mixed-radix Gray-code order (dyadic split with energy cap) do
11:      Apply asymmetry pruning to  $C$ 
12:      if not asymmetry-pruned then
13:        Re-try the previous killing window; if needed, scan all windows of  $C$  in profile-guided
        order using the lookup-table threshold
14:        if some window exceeds the dynamic threshold then
15:          Prune  $C$ 
16:        else
17:           $C$  survives; canonicalize and store
18:        end if
19:      end if
20:    end for
21:  end for
22:  Deduplicate survivors (canonicalize  $\rightarrow$  sort  $\rightarrow$  scan)
23:   $\text{survivors}_k \leftarrow$  deduplicated survivors
24:  if  $|\text{survivors}_k| = 0$  then
25:    return ZERO-SURVIVORS
26:  end if
27: end for
28: return INCONCLUSIVE:  $|\text{survivors}_{L_{\text{max}}}|$  compositions remain

```

7.2. Proof of the Main Theorem

Proof of Theorem 1.2. Normalize

$$g = \frac{f}{\int f},$$

so that $\int g = 1$. The autoconvolution ratio is scale-invariant, so it suffices to prove $R(g) \geq \frac{7}{5}$.

At parameters $(n, m) = (64, 20)$, the identity `sum_bin_masses_eq_one` gives total bin mass 1, so the cumulative-floor construction `canonical_discretization` produces a composition $c \in \mathbb{N}^{128}$ with

$\sum_i c_i = 20$. By the terminal certificate Theorem 6.3, there exist (ℓ, s_{10}) such that

$$\text{TV}_{64,20}(c; \ell, s_{10}) > \frac{7}{5} + \frac{4 \cdot 64}{\ell} \left(\frac{1}{20^2} + \frac{2}{20^2} \sum_{i \in \mathcal{B}_{64, \ell, s_{10}}} c_i \right).$$

This is exactly the hypothesis of Theorem 3.6 (`dynamic_threshold_sound_w_refined` in Lean), whose proof combines the continuous lower bound `continuous_test_value_le_ratio` with the per-window discretization estimate from Lemma 3.4. Therefore $R(g) \geq \frac{7}{5}$, and scale invariance gives $R(f) \geq \frac{7}{5}$.

Taking the infimum over $f \in \mathcal{F}$ yields $C_{1a} \geq \frac{7}{5}$. $\square$

8. Soundness of the Computation

The external computation matters only through the finite statement of Theorem 6.3. To justify importing that statement, one must verify that the implementation tested every relevant terminal composition against the correct threshold. We isolate five soundness conditions and describe how each is verified.

8.1. Complete Enumeration

Every integer composition $c \in \mathbb{N}^d$ with $\sum_i c_i = m$ must be either directly checked or covered by a checked ancestor and all its refinements. At L_0 , the generator produces all $\binom{m+d_0-1}{d_0-1}$ compositions (verified against the stars-and-bars formula for all parameter choices up to $d = 16$, $m = 20$), up to the reversal symmetry from Section 4.3; at L_k ($k \geq 1$), all children of all surviving parents are generated with no sampling. Canonical enumeration counts were checked against the exact count $\frac{N+P}{2}$ from Proposition 5.5.

8.2. Window Exhaustion

For each composition not eliminated by asymmetry, every valid window (ℓ, s_{10}) with $2 \leq \ell \leq 2d$ and all valid starting positions s_{10} is tested before the composition is declared a survivor. An independent reference implementation agrees with the optimized kernel on all 1,771 canonical compositions at $(d, m) = (4, 20)$.

8.3. Conservative Arithmetic

The autoconvolution is computed in exact integer arithmetic. Multiplying both sides of the inequality in Theorem 3.6 by $m^2 \ell / (4n)$, the code compares the integer left-hand side

$$\sum_{s=s_{10}}^{s_{10}+\ell-2} \sum_{i+j=s} c_i c_j = \text{TV}_{n,m}(c; \ell, s_{10}) \frac{m^2 \ell}{4n}$$

against the right-hand side

$$c_{\text{target}} m^2 \frac{\ell}{4n} + 1 + 2W_{\text{int}},$$

with an added $10^{-9} m^2$ margin and a final $(1 - 4\epsilon_{\text{mach}})$ factor before flooring. The multiplicative factor lowers the threshold, but the additive margin dominates, so the net effect is conservative: the code may miss a borderline prune but never falsely prunes.

8.4. Refinement Completeness

Every refinement of a surviving parent is checked at the finer level. The dyadic splitting with energy cap generates all valid children (Section 6.3); children exceeding the cap are provably pruned, by Proposition 6.1 the L^∞ norm of any admissible f with that bin pattern already exceeds the target, so no further refinement is required for those children. The Gray-code kernel produces identical canonical survivor sets as the earlier lexicographic kernel on all tested parents.

8.5. Terminal Cascade

The cascade reached zero survivors at level L_5 ($d = 4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 128$) after approximately 70 hours on a 128-core machine. In the Lean formalization this outcome is packaged as the axiom `cascade_all_pruned`: every composition in $\mathbb{N}^{128}$ of total mass 20 has some window whose test value exceeds the corrected threshold at target $\frac{7}{5}$. The universal quantifier over $\mathbb{N}^{128}$ rather than over canonical representatives is justified by Theorem 4.3: for each non-canonical c the witness for `rev(c)` transports to a witness for c with the same width ℓ and reflected window start, so the canonical coverage produced by the cascade extends to the full lattice.

9. Lean Formalization

The analytic reduction underlying Theorem 1.2 is formalized in Lean 4 [dMU21] on top of the Mathlib library [The20], under `lean/Sidon/`. The development factors the reduction into:

- the per-window continuous lower bound (Lemma 2.3),
- the per-window discretization error estimate (Lemma 3.4),
- the dynamic-threshold soundness theorem (Theorem 3.6, in its CS-form and its W -refined form),
- a parametric bridge `cascade_pruned_implies_bound` that converts a cascade-pruned certificate at any target c_{target} into a lower bound $R(f) \geq c_{\text{target}}$.

Because `autoconvolution_ratio` is encoded using `ENNReal.toReal`, the formal theorem statements carry an explicit finiteness hypothesis on the essential supremum of $f * f$; this hypothesis is harmless for the lower bound since the alternative $\|f * f\|_{L^\infty} = \infty$ already exceeds every finite target vacuously.

Current verified instantiation. The chain is currently instantiated at parameters $(n_{\text{half}}, m, c_{\text{target}}) = (2, 20, 32/25)$, the published Cloninger–Steinerberger baseline, producing the theorem `autoconvolution_ratio_ge_32_25` in `FinalResult.lean`. This instantiation imports a single computational axiom `cascade_all_pruned` that encodes the multi-level cascade as a recursive predicate `CascadePruned`: a composition at resolution d is cascade-pruned if either some window directly exceeds the threshold, or every valid child at resolution $2d$ is cascade-pruned. The axiom is reproducible by the command recorded in `FinalResult.lean`.

Status of the $\frac{7}{5}$ bound. The cascade run reported in Section 8 terminates with zero survivors at $d = 128$ for $c_{\text{target}} = \frac{7}{5}$ using the dynamic per-window correction. Promoting this run into a Lean theorem only requires regenerating the `cascade_all_pruned`-style axiom at the stronger target and composing it with `dynamic_threshold_sound_w_refined` and `cascade_pruned_implies_bound`, which are already kernel-checked. This last step is the only component of Theorem 1.2 that has not yet been replayed inside the Lean kernel; the cascade certificate itself is independently checked at the Python level (see Section 8).

10. Discussion

10.1. Relationship to the Original CS17 Proof

The framework here directly extends Cloninger–Steinerberger [CS17]. The principal differences:

1. **Dynamic correction:** We use a sharper per-window correction rather than only a coarse global bound. This improves pruning while preserving rigor.
2. **Multiscale cascade:** CS17 used $m = 50$ and refined from $n = 3$ through $n = 24$. We use $m = 20$ with dyadic refinement from $d = 4$ through $d = 128$.
3. **Fused generation:** each child is generated and tested inline, reducing per-parent memory from $O(N_{\text{children}} \cdot d)$ to $O(d)$.

Paper statement	Lean name	File
Lemma 2.1	sum_bin_masses_eq_one	TestValueBounds.lean
Lemma 2.3	continuous_test_value_le_ratio	TestValueBounds.lean
Lemma 3.4 (CS form)	correction_term_bound_cs	DiscretizationError.lean
Lemma 3.4 (W-refined)	correction_term_bound_w_refined	WRefinedBound.lean
Theorem 3.6 (CS)	dynamic_threshold_sound_cs	DiscretizationError.lean
Theorem 3.6 (W-refined)	dynamic_threshold_sound_w_refined	WRefinedBound.lean
Parametric bridge	cascade_pruned_implies_bound	FinalResult.lean
Theorem 6.3 (axiom)	cascade_all_pruned	FinalResult.lean
Baseline instantiation	autoconvolution_ratio_ge_32_25	FinalResult.lean

Table 2: Correspondence between paper statements and the Lean formalization; file paths are relative to lean/Sidon/Proof/. The files listed are checked by the Lean kernel; the only imported component is cascade_all_pruned, the computational certificate whose current instantiation matches the $\frac{32}{25}$ baseline. The same scaffolding instantiates the $\frac{7}{5}$ headline bound once the cascade certificate is replayed at the stronger target.

4. **Integer arithmetic:** Autoconvolution and threshold comparisons use exact integer operations, with floating-point involved only in the conservative threshold computation.

10.2. Limitations

- The proof is *computer-assisted*: the mathematical argument reduces correctness to a finite computation, but that computation must be performed by a machine. This paradigm is shared by other recent results where a finite but infeasible-by-hand verification completes a mathematical argument, notably the formal proof of the Kepler conjecture [HAB⁺17] and the numerical verification of the Riemann hypothesis up to $3 \cdot 10^{12}$ [PT21].
- The approach is limited by the exponential growth of the composition count with d and m . The multiscale cascade mitigates this, but very aggressive targets (c_{target} close to C_{1a}) will require more cascade levels and higher computational cost.
- The Lean development currently certifies the $\frac{32}{25}$ baseline; the $\frac{7}{5}$ headline bound requires regenerating the cascade_all_pruned-style axiom at the stronger target (Section 9). The parametric chain that consumes the axiom is already kernel-checked, and the analogous update at any pruned target is mechanical. Eliminating the computational axiom entirely would require a kernel-level certification of the same multi-day search.
- The formal statement isolates the case $\|f * f\|_{L^\infty} < \infty$ because of the way the ratio is encoded in Lean. This is harmless for the lower bound on C_{1a} , since functions with unbounded self-convolution already satisfy every finite lower bound vacuously.

Acknowledgments

This work builds on the windowed autoconvolution lower-bound framework of Cloninger and Steinerberger [CS17], the simplex grid analysis of Bomze, Gollwitzer, and Yildirim [BGY14], and the earlier lower-bound progression by Martin and O’Bryant [MO07, MO09] and Matolcsi and Vinuesa [MV10]. The Lean formalization is built on the Lean 4 toolchain [dMU21] and the Mathlib library [The20], whose authors and contributors deserve particular thanks.

References

- [BGY14] Immanuel M. Bomze, Stefan Gollowitzer, and E. Alper Yildirim, *Rounding on the standard simplex: Regular grids for global optimization*, Journal of Global Optimization **59** (2014), no. 2–3, 243–258.
- [BL26] Christopher Boyer and Zane Kun Li, *An improved example for an autoconvolution inequality*, Experimental Mathematics (2026), Published online 15 February 2026.
- [CRV10] Javier Cilleruelo, Imre Z. Ruzsa, and Carlos Vinuesa, *Generalized sidon sets*, Advances in Mathematics **225** (2010), no. 5, 2786–2807.
- [CS17] Alexander Cloninger and Stefan Steinerberger, *On suprema of autoconvolutions with an application to sidon sets*, Proceedings of the American Mathematical Society **145** (2017), no. 8, 3191–3200.
- [CV08] Javier Cilleruelo and Carlos Vinuesa, *$B_2[g]$ sets and a conjecture of schinzel and schmidt*, Combinatorics, Probability and Computing **17** (2008), no. 6, 741–747.
- [dMU21] Leonardo de Moura and Sebastian Ullrich, *The lean 4 theorem prover and programming language*, Automated Deduction – CADE 28, Lecture Notes in Computer Science, vol. 12699, Springer, Cham, 2021, pp. 625–635.
- [ET41] P. Erdős and P. Turán, *On a problem of sidon in additive number theory, and on some related problems*, Journal of the London Mathematical Society **s1-16** (1941), no. 4, 212–215.
- [HAB⁺17] Thomas Hales, Mark Adams, Gertrud Bauer, Tat Dat Dang, John Harrison, Le Truong Hoang, Cezary Kaliszyk, Victor Magron, Sean McLaughlin, Tat Thang Nguyen, Quang Truong Nguyen, Tobias Nipkow, Steven Obua, Joseph Pleso, Jason Rute, Alexey Solovyev, Thi Hoai An Ta, Nam Trung Tran, Thi Diep Trieu, Josef Urban, Ky Vu, and Roland Zumkeller, *A formal proof of the Kepler conjecture*, Forum of Mathematics, Pi **5** (2017), e2.
- [MO06] Greg Martin and Kevin O’Bryant, *Constructions of generalized sidon sets*, Journal of Combinatorial Theory, Series A **113** (2006), no. 4, 591–607.
- [MO07] ———, *The symmetric subset problem in continuous ramsey theory*, Experimental Mathematics **16** (2007), no. 2, 145–165.
- [MO09] ———, *The supremum of autoconvolutions, with applications to additive number theory*, Illinois Journal of Mathematics **53** (2009), no. 1, 219–235.
- [MV10] Mate Matolcsi and Carlos Vinuesa, *Improved bounds on the supremum of autoconvolutions*, Journal of Mathematical Analysis and Applications **372** (2010), no. 2, 439–447.
- [PT21] David J. Platt and Timothy S. Trudgian, *The Riemann hypothesis is true up to $3 \cdot 10^{12}$* , Bulletin of the London Mathematical Society **53** (2021), no. 3, 792–797.
- [SS02] Andrzej Schinzel and Wolfgang M. Schmidt, *Comparison of L^1 - and L^∞ -norms of squares of polynomials*, Acta Arithmetica **104** (2002), no. 3, 283–296.
- [Tao] Terence Tao, *An autocorrelation constant related to sidon sets*, <https://teorth.github.io/optimizationproblems/constants/1a.html>, Accessed 2026-05-12.

- [The20] The mathlib Community, *The lean mathematical library*, Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (New Orleans, LA, USA), 2020, pp. 367–381.
- [Whi23] Ethan Patrick White, *A new bound for Erdős’ minimum overlap problem*, Acta Arithmetica **208** (2023), 235–255, Published online 16 August 2023; preprint arXiv:2201.05704.